FADENA
FACULTAD DE DEFENSA NACIONAL

Universidad de la Defensa Nacional

Facultad de Defensa Nacional

Licenciatura en Ciberdefensa

Sistemas de Tratamiento de Datos

Estructuras de datos

Año: 2025

Tipos de Datos

Generalmente, los datos se clasifican en tres categorías principales: estructurados, semi-estructurados y no estructurados.

Datos Estructurados

Los datos estructurados son el tipo de datos más tradicional y organizado. Se ajustan a un modelo de datos predefinido y residen en un formato tabular con filas y columnas bien definidas. Cada pieza de dato tiene un identificador claro, y su significado es explícito debido a su organización.

Características Clave:

- Formato Fijo y Predefinido: Se adhieren a un esquema rígido.
- Fácilmente Almacenables y Recuperables: Ideales para bases de datos relacionales.
- Consultables mediante SQL (structured query language): Lenguajes de consulta estructurados como SQL son perfectos para manipularlos.
- Altamente Analizables: Facilitan el análisis, los informes y la creación de cuadros de mando.

Ejemplos Comunes:

- Bases de Datos Relacionales (RDBMS): Tablas, columnas, filas, claves, relaciones, esquemas, restricciones, índices, etc.
- Hojas de Cálculo: Celdas, filas, columnas, hojas de trabajo, fórmulas, funciones, etc.

- Formularios Web: Campos como nombre, email, fecha de nacimiento, código postal.
- Datos de Sensores IoT: Un sensor IoT (Internet de las Cosas) puede proporcionar diversos tipos de datos dependiendo de su función y aplicación, algunos ejemplos pueden ser: temperatura, humedad, presión, aceleración, ubicación geográfica, proximidad, peso, etc.

Ventajas: Fácil de gestionar, optimizado para rendimiento en consultas específicas, alta integridad de datos.

Desventajas: Poca flexibilidad para cambios de esquema, no apto para datos de naturaleza variada.

Datos Semi-estructurados

Los datos **semi-estructurados** no se ajustan a un modelo de datos rígido como los estructurados, pero contienen etiquetas u otros marcadores que organizan los datos y proporcionan una jerarquía. Tienen cierta organización, pero no están limitados por el formato fijo de una tabla relacional. Permiten una mayor flexibilidad en la estructura y el contenido.

- **Características Clave:**
  - **Formato Flexible pero Organizado:** Utilizan etiquetas para definir elementos y atributos.
  - **Jerárquicos:** A menudo presentan una estructura de árbol o grafo.

- **No Requieren un Esquema Predefinido:** Permiten añadir o eliminar campos sin romper el esquema general.
- **Lectura Humana y Máquina:** Son legibles tanto por personas como por programas de software.

- **Ejemplos Comunes:**
  - **JSON (JavaScript Object Notation):** Muy utilizado en APIs web para intercambio de datos.

```
{
  "nombre": "Juan",
  "edad": 30,
  "ciudad": "Buenos Aires",
  "intereses": ["programacion", "lectura"],
  "contacto": {
    "email": "juan@ejemplo.com",
    "telefono": "1123456789"
  }
}
```

  - **XML (Extensible Markup Language):** Utilizado para documentos web, intercambio de datos entre sistemas.

```
<?xml version="1.0" encoding="UTF-8"?>
<person>
  <nombre>Juan</nombre>
  <edad>30</edad>
  <ciudad>Buenos Aires</ciudad>
  <intereses>
    <interes>programacion</interes>
    <interes>lectura</interes>
  </intereses>
  <contacto>
    <email>juan@ejemplo.com</email>
```

```
        <telefono>1123456789</telefono>
    </contacto>
</person>
```

- YAML (YAML Ain't Markup Language): Configuración de archivos.
- CSV con Encabezados: Si bien es simple, los encabezados dan una estructura básica.
- Logs de Servidor: Pueden tener patrones reconocibles, pero sin un esquema estricto.

Ventajas: Mayor flexibilidad que los estructurados, fácil intercambio de datos, ideal para datos jerárquicos.

Desventajas: Más complejos de consultar que los estructurados (requieren parsers), el significado puede depender del contexto.

Datos No Estructurados

Los **datos no estructurados** no tienen un modelo de datos predefinido ni un formato fijo. Constituyen la mayor parte de los datos generados hoy en día y son los más difíciles de procesar y analizar de forma tradicional. Su significado se deriva de su contenido y a menudo requiere técnicas avanzadas de procesamiento de lenguaje natural (NLP), visión por computadora o aprendizaje automático para extraer valor.

- **Características clave:**

  Sin Estructura Interna Fija: No hay esquema, filas, columnas o marcadores predefinidos.

  - **Texto y Multimedia:** Predominantemente texto, imágenes, audio y video en formatos digitales variados.
  - **Difícil de Consultar Tradicionalmente:** No se pueden usar lenguaje de consulta SQL directamente; requieren análisis de contenido.
  - **Gran Volumen y Variedad:** Contribuyen significativamente al concepto de "Big Data".
- **Ejemplos Comunes:**
  - **Documentos de Texto:** Correos electrónicos, documentos de texto enriquecido (docx, rtf, etc), PDF, notas de texto.
  - **Contenido Multimedia:** Imágenes, videos, archivos de audio (música, grabaciones de voz).
  - **Redes Sociales:** Publicaciones en Twitter, Facebook, comentarios, historias.
  - **Contenido web:** Páginas web, redes sociales, etc.
  - **Datos de Sensores Complejos:** Datos sin procesar de cámaras de seguridad, grabaciones de sonido ambiental.

Ventajas: Captura una riqueza de información que no encaja en formatos estructurados, flexible para nuevos tipos de datos.

Desventajas: Difícil de almacenar (a menudo en data lakes), requiere técnicas avanzadas para el procesamiento y análisis, alto costo computacional para extraer insights.

A medida que el ecosistema de datos evoluciona, las líneas entre estos tipos de datos a menudo se difuminan. Es común que las organizaciones conviertan datos no estructurados en semi-estructurados (ej., extrayendo entidades de un texto a un JSON) o semi-estructurados en estructurados (ej., aplanando un JSON a una tabla).

Estructuras de datos

La importancia de la computación radica fundamentalmente en su capacidad para procesar información. Con el propósito de que la información sea procesada, se requiere que ésta se almacene en la memoria de la computadora. De acuerdo con la forma en que los datos se organizan, se clasifican en:

Tipos de datos simples (primitivos o básicos): La principal característica de los tipos de datos simples consiste en que ocupan sólo una casilla de memoria; por tanto, una variable simple hace referencia a un único valor a la vez. En este grupo de datos se encuentran: números enteros y reales, caracteres, booleanos, etc.

Tipos de datos estructurados (no primitivos): Por otra parte, los tipos de datos estructurados se caracterizan por el hecho de que con un nombre -identificador de variable estructurada- se hace referencia a un grupo de casillas de memoria. Es decir, un tipo de dato estructurado tiene varios componentes. Cada uno de éstos puede ser un tipo de dato simple o estructurado. Sin embargo, los componentes básicos, los del nivel más bajo, de cualquier tipo de datos estructurado son siempre tipos de datos simples.

Las estructuras de datos son métodos organizativos que permiten almacenar, recuperar y manipular datos de manera eficiente en informática. Son fundamentales para la programación y el desarrollo de aplicaciones, ya que facilitan la creación de algoritmos y la gestión de la información. Estas estructuras pueden ser:

Lineales: Organizan los elementos secuencial o linealmente, donde cada elemento tiene un predecesor y un sucesor únicos. Ejemplos de estructuras de datos lineales son arreglos, listas enlazadas, pilas, colas, etc.

No lineales: Organizan los elementos de manera jerárquica o en red, donde cada elemento puede tener múltiples predecesores y sucesores. Ejemplos de estructuras de datos no lineales son árboles, grafos, etc.

Diferentes estructuras de datos almacenan la información de diferente manera, cambiando el espacio que utilizan y la eficiencia al agregar o extraer y consultar, y cómo se relacionan los datos entre sí. Esto no implica que una sea mejor que la otra, dependerá de las necesidades.

Arreglos

Un arreglo (array) es una colección finita y ordenada de elementos del mismo tipo de datos, almacenados en forma contigua dentro de la memoria y accesibles a través de un índice.

Se puede pensar en un arreglo como una fila de casilleros numerados, donde cada casillero contiene un elemento.

Por lo general son de tamaño fijo, se tiene que indicar cuantos elementos va a tener y de qué tipo. Para acceder se usa un índice, que generalmente inicia en 0. Como el tamaño de un arreglo es fijo, es muy fácil para la máquina localizar un elemento en posición n, multiplicando el tamaño del tipo de elemento (que miden lo mismo en memoria) por su índice. Por lo que acceder a un elemento dentro del arreglo para modificarlo u obtener su valor supone una complejidad constante, es casi inmediato. Pero eliminar elementos o agregar elementos modificando el tamaño del arreglo es muy difícil, teniendo que crear un nuevo arreglo con tamaño diferente y copiar los elementos del arreglo original en este, siendo operaciones muy costosas.

Así como podemos tener arreglos de enteros o de strings, se pueden tener arreglos de otros tipos de datos (por ejemplo de enteros). Estos arreglos multidimensionales permiten trabajar con otras estructuras más interesantes como tablas o vectores de información.

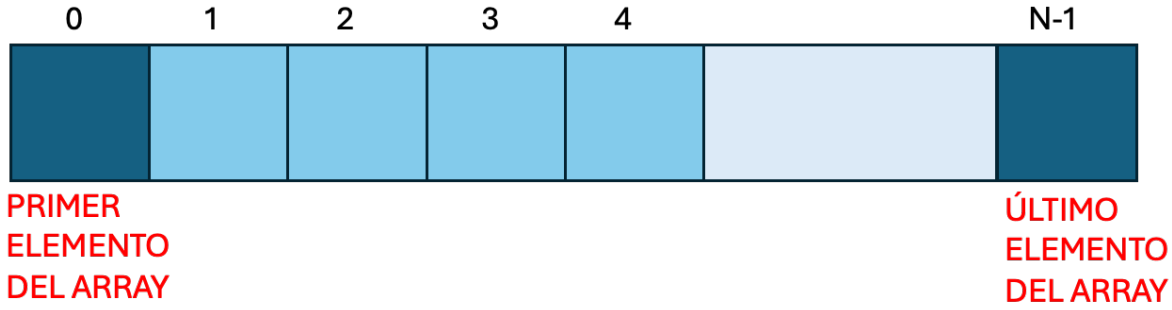
Un **arreglo unidimensional** se define como una colección finita, homogénea y ordenada de elementos.

Finita (tamaño fijo): todo arreglo tiene un límite; es decir, se debe determinar cuál será el número máximo de elementos que formarán parte del arreglo.

Homogénea: todos los elementos de un arreglo son del mismo tipo. Es decir, todos enteros, todos booleanos, etcétera, pero nunca una combinación de distintos tipos.

Ordenada: se puede determinar cuáles son el primero, el segundo, el tercero, … y el enésimo elemento. En este orden los elementos de un

arreglo se guardan en bloques adyacentes de memoria, característica clave para el acceso rápido y eficiente.



Acceso directo por índice: Si un arreglo tiene la característica de que puede almacenar a N elementos del mismo tipo, entonces deberá permitir la recuperación de cada uno de ellos. Como consecuencia, se distinguen dos partes fundamentales en los arreglos:

Los componentes: hacen referencia a los elementos que forman el arreglo; es decir, a los valores que se almacenan en cada una de sus casillas.

Los índices: especifican cuántos elementos tendrá el arreglo y además de qué modo podrán recuperarse esos componentes. Los índices también permiten hacer referencia a los componentes del arreglo en forma individual. Por tanto, para hacer referencia a un elemento de un arreglo se debe utilizar:

- El nombre del arreglo.
- El índice del elemento.

Operaciones básicas con arreglos: Como los arreglos se utilizan para almacenar datos, resulta necesario leer, escribir, insertar, modificar, eliminar, obtener la cantidad total de elementos, ordenar, etc.

- **Acceso a elementos:** Obtener el valor de un elemento en una posición específica usando su índice (por ejemplo, arreglo[2] para acceder al tercer elemento).
- **Inserción:** Agregar un nuevo elemento en una posición determinada o al final del arreglo (por ejemplo, agregar el valor 5 al final: arreglo.append(5) en Python).
- **Eliminación:** Remover un elemento de una posición específica o del final (por ejemplo, eliminar el elemento en el índice 1: arreglo.pop(1)).
- Modificación: Cambiar el valor de un elemento en una posición dada (por ejemplo, arreglo[0] = 10 para cambiar el primer elemento a 10).
- **Búsqueda:** Encontrar la posición de un elemento específico (por ejemplo, arreglo.index(20) para obtener el índice donde está el valor 20).
- Recorrido: Iterar sobre todos los elementos para procesarlos (por ejemplo, usando un bucle for en Python: for elemento in arreglo:).
- **Ordenamiento:** Reorganizar los elementos en orden ascendente o descendente (por ejemplo, arreglo.sort() en Python).
- Filtrado: Crear un nuevo arreglo con elementos que cumplan una condición (por ejemplo, [x for x in arreglo if x > 10] para obtener valores mayores a 10).
- **Concatenación:** Combinar dos arreglos en uno solo (por ejemplo, arreglo1 + arreglo2 en Python).
- **Longitud:** Determinar la cantidad de elementos en el arreglo (por ejemplo, len(arreglo) en Python).

Un **arreglo multidimensional - N dimensiones - (o matriz)** es una colección homogénea, finita y ordenada de datos de dos o más dimensiones. Para hacer referencia a cada componente de un arreglo de N dimensiones, se usarán N índices, uno para cada dimensión. El caso más común es la de dos dimensiones, donde el primer índice se utiliza para indicar el renglón, y el segundo para señalar la columna. Un arreglo bidimensional también se puede definir como un arreglo de arreglos.

	Col 1	Col 2	Col 3	Col 4
Fila 1	Arr[0,0]	Arr[0,1]	Arr[0,2]	Arr[0,3]
Fila 2	Arr[1,0]	Arr[1,1]	Arr[1,2]	Arr[1,3]
Fila 3	Arr[2,0]	Arr[2,1]	Arr[2,2]	Arr[2,3]
Fila 4	Arr[3,0]	Arr[3,1]	Arr[3,2]	Arr[3,3]

Nota: La mayoría de los lenguajes de programación comienzan su índice en 0, por lo que la primera fila o columna suele ser referenciada con el índice 0.

Ventajas de los Arreglos

- **Acceso Rápido:** La principal ventaja es la capacidad de acceder a cualquier elemento de forma instantánea usando su índice. Esto es gracias a la contigüidad en memoria.
- **Eficiencia en memoria:** Al almacenar elementos de forma contigua, los arreglos pueden ser muy eficientes en el uso de la memoria, especialmente si el tamaño se conoce de antemano.

- **Simplicidad:** Son una de las estructuras de datos más sencillas de entender e implementar.
- **Base para otras estructuras:** Muchos algoritmos fundamentales (como ordenamiento y búsqueda binaria) y estructuras de datos (pilas, colas, tablas hash) se basan en arreglos.

Desventajas de los Arreglos

- **Tamaño Fijo (en muchos lenguajes):** La mayor limitación. Si necesitas más espacio, debes crear un nuevo arreglo más grande y copiar los elementos, lo cual es costoso.
- **Inserciones y eliminaciones costosas:** Insertar un elemento en medio de un arreglo requiere desplazar todos los elementos posteriores para hacer espacio. Eliminar un elemento también requiere desplazar elementos para llenar el hueco.
- **Posible desperdicio de memoria:** Si se declara un arreglo muy grande para prever futuras necesidades, pero no se utiliza todo el espacio, se desperdicia memoria.

Aplicaciones comunes de los Arreglos

- **Almacenar listas de elementos:** Nombres, calificaciones, productos en un inventario, etc.
- **Representar matrices y vectores:** En matemáticas y gráficos por computadora.
- **Procesamiento de Imágenes:** Las imágenes se pueden representar como matrices de píxeles.
- **Tablas de Datos:** Similares a una hoja de cálculo.

- **Implementación de otras estructuras de datos:** Pilas (stacks), colas (queues), tablas hash, árboles binarios (en algunos casos).
- **Desarrollo de Videojuegos:** Mapas de juegos, posiciones de personajes, puntajes, etc.

Listas Enlazadas

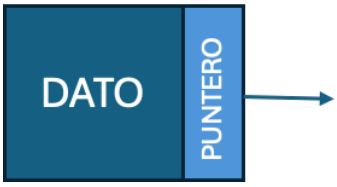
Las listas son estructuras lineales y dinámicas de datos. La principal ventaja del dinamismo lo representa el hecho de que se adquieren posiciones de memoria a medida que se necesitan y se liberan cuando ya no se requieren. Es decir, se llegan a expandir o contraer, dependiendo de la aplicación. El dinamismo de estas estructuras soluciona el problema de decidir cuánto espacio se necesita a priori, por ejemplo, en una estructura de datos estática como el arreglo.

A diferencia de los arreglos (arrays), que almacenan elementos en ubicaciones de memoria contiguas, las listas enlazadas almacenan elementos en ubicaciones de memoria dispersas, utilizando punteros (o referencias) para mantener el orden.

La unidad fundamental de una lista enlazada es el **nodo**. Cada nodo típicamente contiene dos partes:

1. **Dato (o Valor):** El elemento que se almacena.
2. **Puntero (o Referencia):** Una referencia al siguiente nodo en la secuencia.

Diagrama conceptual de un nodo:



Lista enlazada simple

Una **Lista Enlazada Simple** (Singly Linked List) es la forma más básica de lista enlazada. Consiste en una secuencia de nodos donde cada nodo apunta al siguiente en la lista. El último nodo apunta a None (o null), indicando el final de la lista.

La lista en sí misma se representa por un puntero al primer nodo, llamado **"cabeza" (head)**. Si la lista está vacía, head apunta a None.

Diagrama conceptual de una Lista Enlazada Simple:



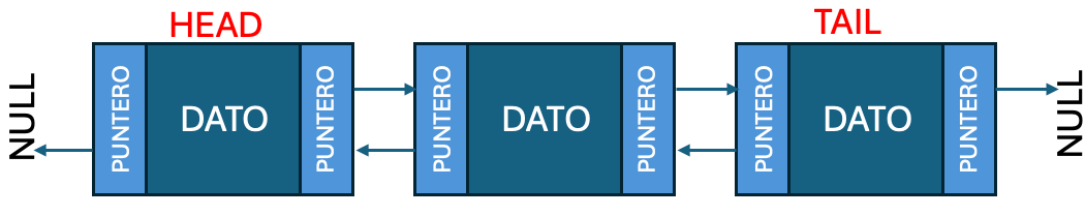
Lista Enlazada Doble

Una **Lista Enlazada Doble** permite la navegación en ambas direcciones (hacia adelante y hacia atrás). Cada nodo tiene tres partes:

1. **Dato (o Valor):** El elemento almacenado.
2. **Puntero al Siguiente:** Referencia al siguiente nodo.
3. **Puntero al Anterior:** Referencia al nodo previo.

La lista en sí misma típicamente mantiene punteros a la **cabeza (head)** y a la **cola (tail)**.

Diagrama conceptual de una Lista Enlazada Doble:



Ventajas:

- Navegación bidireccional.
- La inserción y eliminación de nodos (dado el nodo) son más eficientes ya que no necesita buscar el nodo anterior.

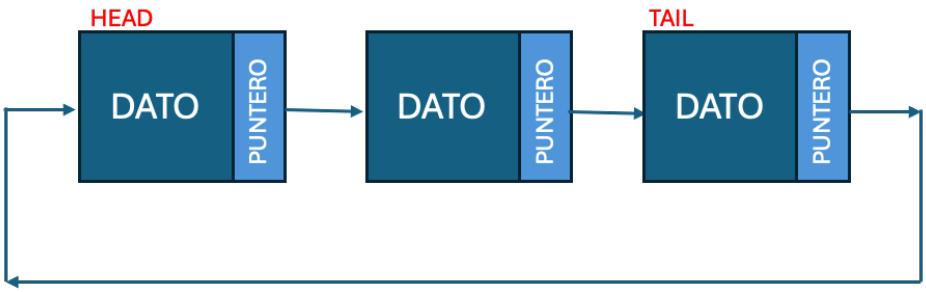
Desventajas:

- Mayor uso de memoria por cada nodo (un puntero adicional).
- Las operaciones de inserción/eliminación son ligeramente más complejas debido a la necesidad de actualizar dos punteros.

Listas Enlazadas Circulares (Circular Linked List)

En una **Lista Enlazada Circular**, el último nodo de la lista apunta al primer nodo, creando un ciclo. Esto elimina la necesidad de None como terminador

Diagrama conceptual de una Lista Enlazada Simple Circular:



Ventajas:

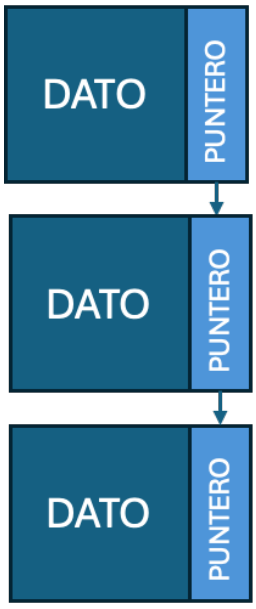
- Permite un recorrido continuo de la lista sin un final.
- Acceso a cualquier nodo desde cualquier otro nodo.
- Útil para implementar estructuras como un "round-robin" scheduler (programador de tareas).

Desventajas:

- Las operaciones pueden ser más complejas ya que no hay un None para indicar el final, y se debe manejar el ciclo cuidadosamente para evitar bucles infinitos.

Pilas

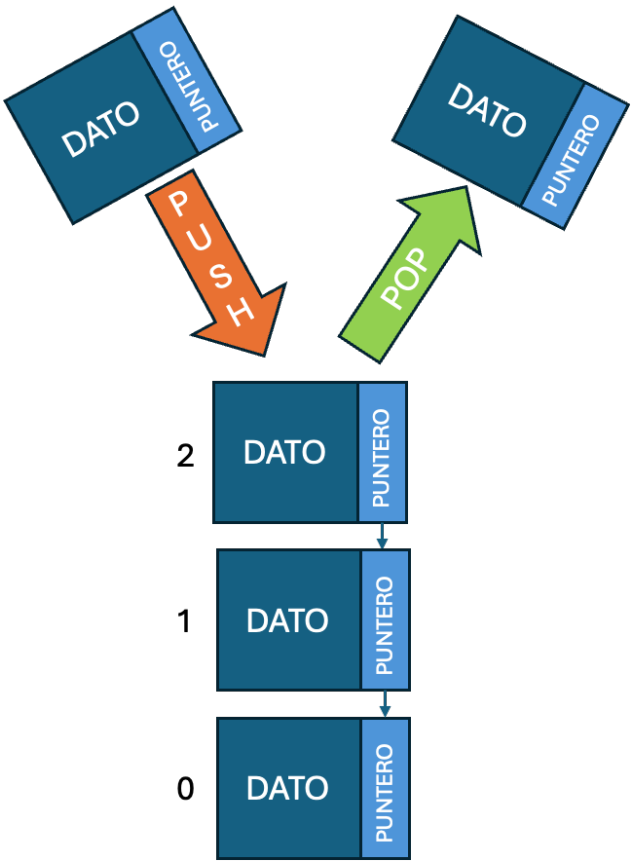
Una pila representa una estructura lineal de datos en la que se puede agregar o quitar elementos únicamente por uno de los dos extremos. En consecuencia, los elementos de una pila se eliminan en orden inverso al que se insertaron; es decir, el último elemento que se mete en la pila es el primero que se saca. Debido a esta característica, se le conoce como estructura **LIFO** (Last-Input, First-Output: el último en entrar es el primero en salir).



Operaciones Básicas:

- **push(elemento)**: Añade un elemento a la parte superior de la pila.
- **pop()**: Elimina y devuelve el elemento superior de la pila. Si la pila está vacía, suele lanzar una excepción o devolver un valor especial (como None).

- **peek()** o **top()**: Devuelve el elemento superior de la pila sin eliminarlo. Si la pila está vacía, suele lanzar una excepción o devolver un valor especial.
- **isEmpty()**: Devuelve True si la pila no contiene elementos, False en caso contrario.
- **size()**: Devuelve el número de elementos en la pila.



Implementación:

Las pilas no son estructuras fundamentales de datos; es decir, no están definidas como tales en los lenguajes de programación. Para su representación requieren el uso de otras estructuras de datos, como: Arreglos o Listas Enlazadas.

Aplicaciones Comunes de las Pilas:

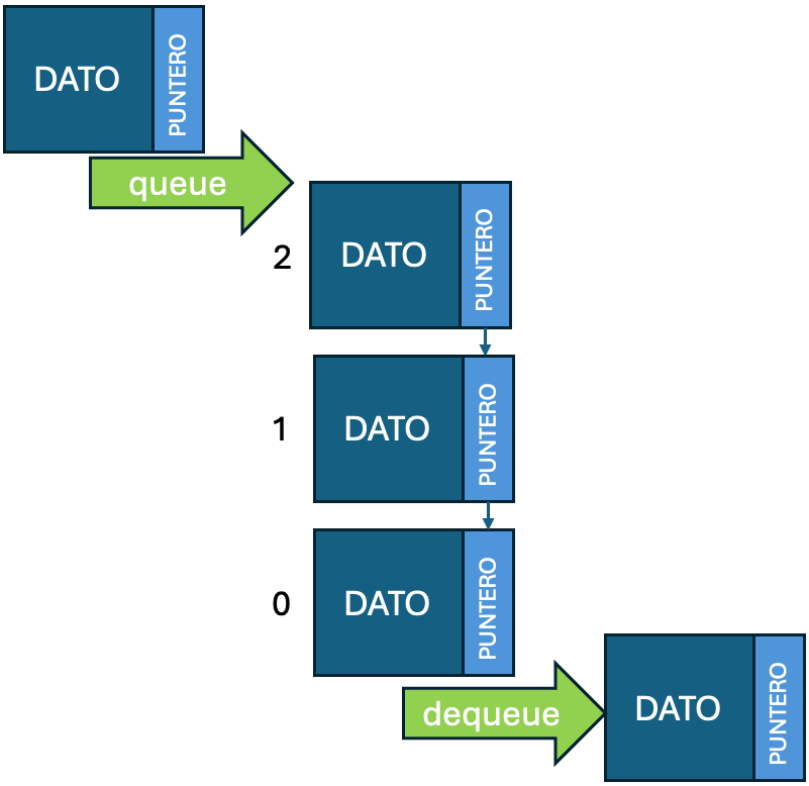
- **Gestión de llamadas a funciones (Call Stack):** Cuando un programa llama a una función, la dirección de retorno y las variables locales se guardan en la pila de llamadas.
- **Deshacer/Rehacer operaciones:** Los programas suelen guardar las acciones realizadas en una pila para poder deshacerlas.
- **Evaluación de expresiones:** Las pilas se usan para convertir y evaluar expresiones matemáticas.
- **Navegación web (historial de páginas visitadas):** El botón "Atrás" del navegador funciona como una pila.
- **Reconocimiento de patrones (parsing de lenguajes):** En compiladores y analizadores sintácticos.
- **Algoritmos de búsqueda en profundidad (DFS - Depth First Search):** En grafos y árboles.

Colas

Una cola constituye una estructura lineal de datos en la que los nuevos elementos se introducen por un extremo y los ya existentes se eliminan por el otro. Es importante señalar que los componentes de la cola se eliminan en el mismo orden en el cual se insertaron. Es decir, el primer elemento que se introduce en la estructura será el que se eliminará en primer orden. Debido a esta característica, las colas también reciben el nombre de estructuras **FIFO** (First-In, First-Out: el primero en entrar es el primero en salir).

Operaciones Básicas:

- **enqueue(elemento)** o **offer(elemento)**: Añade un elemento al final (parte trasera) de la cola.
- **dequeue()** o **poll()**: Elimina y devuelve el elemento del principio (parte frontal) de la cola. Si la cola está vacía, suele lanzar una excepción o devolver un valor especial.
- **front()** o **peek()**: Devuelve el elemento del principio de la cola sin eliminarlo. Si la cola está vacía, suele lanzar una excepción o devolver un valor especial.
- **isEmpty()**: Devuelve True si la cola no contiene elementos, False en caso contrario.
- **size()**: Devuelve el número de elementos en la cola.



Implementación:

Al igual que las pilas, las colas no son estructuras fundamentales de datos; es decir, no están definidas como tales en los lenguajes de programación. Para su representación requieren el uso de otras estructuras de datos, como: Arreglos o Listas Enlazadas.

Aplicaciones Comunes de las Colas:

- **Gestión de tareas en sistemas operativos:** Planificación de procesos, impresión, etc.
- **Buffering de datos:** En sistemas de comunicación, redes, streaming.
- **Simulaciones:** Para modelar situaciones de la vida real (colas de espera, eventos).
- **Algoritmos de búsqueda en anchura (BFS - Breadth First Search):** En grafos y árboles.
- **Gestión de peticiones en servidores web:** Las peticiones entrantes se ponen en cola para ser procesadas.

Recursión

La recursión es una técnica de programación donde una **función se llama a sí misma** para resolver un problema. Esta auto-llamada continúa hasta que se alcanza una condición específica, conocida como **caso base**, que permite que la función retorne un valor sin hacer más llamadas recursivas.

Componentes clave de una función recursiva:

Cada función recursiva debe tener dos partes fundamentales:

1. **Caso Base (Base Case):** Es la condición de terminación de la recursión. Cuando se cumple el caso base, la función devuelve un valor directamente sin hacer más llamadas recursivas. Sin un caso base adecuado, la recursión se ejecutaría infinitamente, llevando a un "desbordamiento de pila" (stack overflow).
2. **Paso Recursivo (Recursive Step):** Es la parte de la función donde se realiza la llamada a sí misma, pero con una versión más pequeña o simplificada del problema original. El problema se reduce en cada llamada hasta que finalmente se llega al caso base.

¿Cómo funciona la Recursión? (La pila de llamadas)

Cuando una función se llama a sí misma (o a cualquier otra función), el sistema operativo utiliza una estructura de datos llamada **pila de llamadas (call stack)**. Cada vez que se hace una llamada a una función, se crea un "marco de pila" (stack frame) que contiene la información de esa llamada (variables locales, dirección de retorno, etc.) y se empuja a la cima de la pila.

En la recursión:

- Cada llamada recursiva empuja un nuevo marco a la pila.
- Cuando se alcanza el caso base, la función actual retorna su valor.
- El marco de esa función se saca de la pila (pop).

- La ejecución regresa al marco de la función que la llamó, utilizando el valor retornado, y así sucesivamente hasta que la llamada original finaliza.

Ejemplos Clásicos de Recursión

Vamos a explorar algunos ejemplos comunes para entender cómo se aplica la recursión.

Ejemplo 1: Factorial de un Número

El factorial de un número entero no negativo n, denotado como n, es el producto de todos los enteros positivos menores o iguales a n.

```
ALGORITMO CalcularFactorialRecursivo
    INICIO
        ESCRIBIR "Ingresa un número entero no negativo: "
        LEER numero
        (SI) numero < 0 ENTONCES
            ESCRIBIR "El factorial no está definido para números negativos"
          (SINO)
            resultado = FactorialRecursivo(numero)
            ESCRIBIR "El factorial de ", numero, " es: ", resultado
        (FIN SI)
 FIN


FUNCIÓN FactorialRecursivo(n)
    INICIO
        (SI) n = 0 ó n = 1 ENTONCES
            RETORNAR 1
        (SINO)
```

```
            RETORNAR n * FactorialRecursivo(n - 1)

        (FIN SI)

    FIN
```

Explicación

- Entrada: El usuario ingresa un número entero no negativo.
- Validación: Se comprueba si el número es negativo; si lo es, se indica que no está definido.
- Llamada recursiva: La función FactorialRecursivo se llama con el número ingresado.
    - Caso base: Si
      $n=0n = 0n = 0$
      o
      $n=1n = 1n = 1$
      , retorna 1, lo que detiene la recursión.
    - Paso recursivo: Si
      $n>1n > 1n > 1$
      , la función se llama a sí misma con
      $n−1n-1n-1$
      , multiplicando el resultado por (n).
- Salida: Se muestra el resultado final.

Ejemplo 2: Números de Fibonacci

La secuencia de Fibonacci es una serie donde cada número es la suma de los dos anteriores, comenzando con 0 y 1.

```
ALGORITMO CalcularFibonacci

    INICIO

   ESCRIBIR "Ingresa un número entero no negativo para la posición en la secuencia
de Fibonacci: "
```

```
        LEER numero

        (SI) numero < 0 ENTONCES

            ESCRIBIR "La posición no puede ser negativa"

        (SINO)

            resultado = FibonacciRecursivo(numero)

            ESCRIBIR "El número de Fibonacci en la posición ", numero, " es: ",
resultado

        (FIN SI)

    FIN


FUNCIÓN FibonacciRecursivo(n)

    INICIO

        (SI) n = 0 ENTONCES

            RETORNAR 0

        (SINO)

            (SI) n = 1 ENTONCES

              RETORNAR 1

        (SINO)

            RETORNAR FibonacciRecursivo(n - 1) + FibonacciRecursivo(n - 2)

        (FIN SI)

    FIN
```

Explicación

- Entrada: El usuario ingresa un número entero no negativo que representa la posición en la secuencia de Fibonacci (por ejemplo, (F(5)) es el sexto número).
- Validación: Se verifica que el número no sea negativo.
- Llamada recursiva: La función FibonacciRecursivo se define así:
    - Casos base: Si

      n=0n = 0n = 0

      , retorna 0; si

n=1n = 1n = 1

, retorna 1.

- Paso recursivo: Para

  n>1n > 1n > 1

  , calcula (F(n)) sumando

  F(n−1)F(n-1)F(n-1)

  y

  F(n−2)F(n-2)F(n-2)

  mediante nuevas llamadas recursivas.

- Salida: Se muestra el valor de Fibonacci en la posición solicitada.

Nota: Aunque el ejemplo de Fibonacci es clásico para ilustrar la recursión, su implementación recursiva directa es **ineficiente** debido a la **repetición de cálculos** (por ejemplo, FibonacciRecursivo(4) se calcula múltiples veces). Esto es un problema de rendimiento que se puede mitigar con técnicas como la memorización o la programación dinámica, o simplemente con una solución iterativa.

Ejemplo 3: Suma de Elementos de una Lista

Sumar todos los elementos de una lista utilizando recursión.

- **Caso Base:** Si la lista está vacía, la suma es 0.
- **Paso Recursivo:** La suma de la lista es el primer elemento más la suma del resto de la lista.

Ventajas y Desventajas de la Recursión

Ventajas:

- **Claridad y Elegancia:** Para problemas que tienen una definición recursiva natural (como los fractales, los recorridos de árboles o la búsqueda en profundidad), una solución recursiva a menudo es más concisa y fácil de leer que una iterativa.
- **Código Conciso:** Puede llevar a un código más corto y menos propenso a errores para ciertos problemas.
- **Resolución de Problemas Complejos:** Algunas estructuras de datos (como árboles y grafos) o algoritmos (como el Quicksort o Mergesort) son inherentemente recursivos y su implementación recursiva es más intuitiva.

Desventajas:

- **Uso de Memoria (Stack Overflow):** Cada llamada recursiva añade un marco a la pila de llamadas. Para problemas con muchas llamadas recursivas (profundidad de recursión grande), esto puede llevar a un "desbordamiento de pila", agotando la memoria disponible. Python tiene un límite de recursión por defecto (generalmente 1000).
- **Ineficiencia de Rendimiento:** La sobrecarga de gestionar la pila de llamadas (empujar y sacar marcos) puede hacer que las soluciones recursivas sean más lentas que sus equivalentes iterativos para algunos problemas.

- **Dificultad de Depuración:** Rastrear el flujo de ejecución en una función recursiva puede ser más complejo debido a las múltiples llamadas y retornos.

Recursión vs. Iteración

Prácticamente cualquier problema que pueda resolverse recursivamente también puede resolverse **iterativamente** (usando bucles como **for** o **while**). La elección entre recursión e iteración a menudo depende de la naturaleza del problema, la claridad del código y las consideraciones de rendimiento.

- **Recursión:**
  - Ventajoso para problemas que se definen inherentemente de forma recursiva (ej. recorridos de árboles, fractales).
  - Puede ser más fácil de entender y escribir para ciertos algoritmos.
  - Riesgo de desbordamiento de pila y posible ineficiencia.
- **Iteración:**
  - Generalmente más eficiente en términos de memoria y rendimiento (no hay sobrecarga de la pila de llamadas).
  - Más control sobre el flujo de ejecución.
  - Puede ser más compleja de escribir para problemas inherentemente recursivos, requiriendo el uso explícito de pilas o colas.

Árboles

Los árboles son las estructuras de datos **no lineales** y **dinámicas** más importantes del área de computación. Dinámicas, porque las mismas pueden cambiar tanto de forma como de tamaño durante la ejecución de un programa. No lineales, porque cada elemento del árbol puede tener

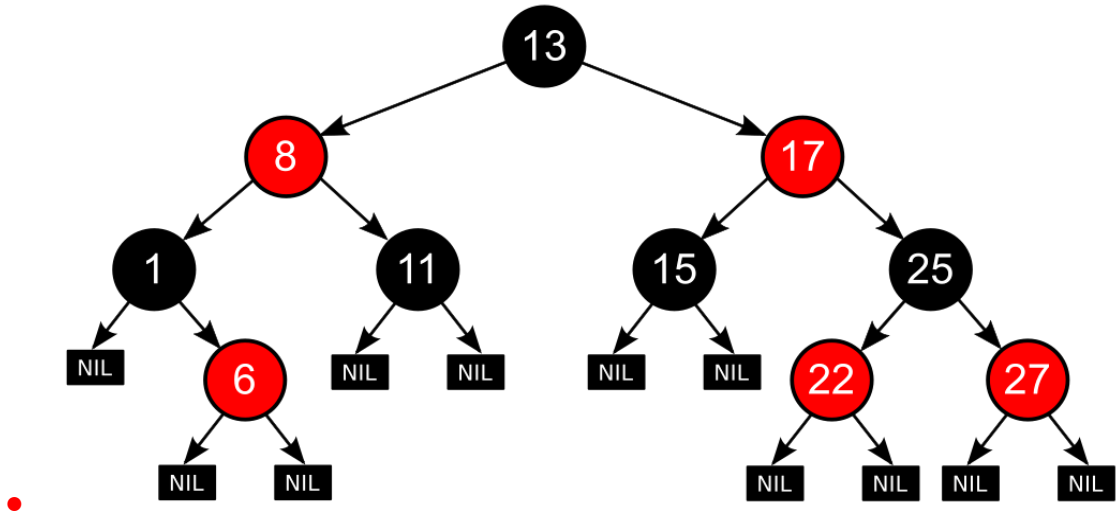
más de un sucesor. A diferencia de las estructuras lineales como Pilas y Colas, donde los elementos están secuencialmente ordenados, los árboles permiten una ramificación, lo que los hace ideales para organizar la información que tiene una jerarquía natural (por ejemplo la estructura de un sistema de archivos en la computadora).

Árboles en general

Un árbol se puede definir como una estructura jerárquica aplicada sobre una colección de elementos u objetos llamados nodos, uno de los cuales es conocido como raíz. Además se crea una relación o parentesco entre los nodos dando lugar a términos como padre, hijo, hermano, antecesor, sucesor, ancestro, etc.

Formalmente se define un árbol de tipo T como una estructura homogénea resultado de la concatenación de un elemento de tipo T con un número finito de árboles disjuntos, llamados subárboles. Una forma particular de árbol es el árbol vacío. Los árboles son estructuras recursivas, ya que cada subárbol es a su vez un árbol.

Se puede representar un árbol a través de un grafo donde se distinguen **nodos** -por ejemplo con círculos- y **arcos** -por ejemplo con líneas con flechas-. Los primeros se usan para almacenar la información, y los últimos para establecer las relaciones entre los nodos.



Características y propiedades de los árboles.

Raíz (Root): El nodo superior de un árbol. No tiene padres. En un árbol no vacío, siempre hay una única raíz.

Nodo (Node): Cada elemento en el árbol.

Hijo (Child): Un nodo directamente conectado debajo de otro nodo. Un nodo X es descendiente directo de un nodo Y, si el nodo X es apuntado por el nodo Y.

Padre (Parent): Un nodo directamente conectado arriba de otro nodo. Un nodo X es antecesor directo de un nodo Y, si el nodo X apunta al nodo Y.

Hermano (Sibling): Nodos que comparten el mismo padre.

Hoja (Leaf): Un nodo que no tiene hijos. Todo nodo que no tiene ramificaciones se conoce como terminal u hoja.

Rama (Branch/Internal Node): Todo nodo que no es raíz ni terminal u hoja se conoce con el nombre de interior.

Borde/Arista (Edge): El enlace entre dos nodos.

Grado (Degree): El número de descendientes directos de un determinado nodo.

Grado del árbol: Es el máximo grado de todos los nodos del árbol.

Nivel (Level): La distancia de un nodo desde la raíz. La raíz está en el nivel 0. Los hijos de un nodo en el nivel k están en el nivel k+1.

Altura (Height): La longitud del camino más largo desde la raíz hasta una hoja. O la altura de un nodo es la longitud del camino más largo desde ese nodo hasta una hoja descendiente. (La altura de un árbol es la altura de su raíz). La altura de un árbol vacío es -1.

Profundidad (Depth): La longitud del camino desde la raíz hasta un nodo específico. La profundidad de la raíz es 0.

Ventajas de los Árboles:

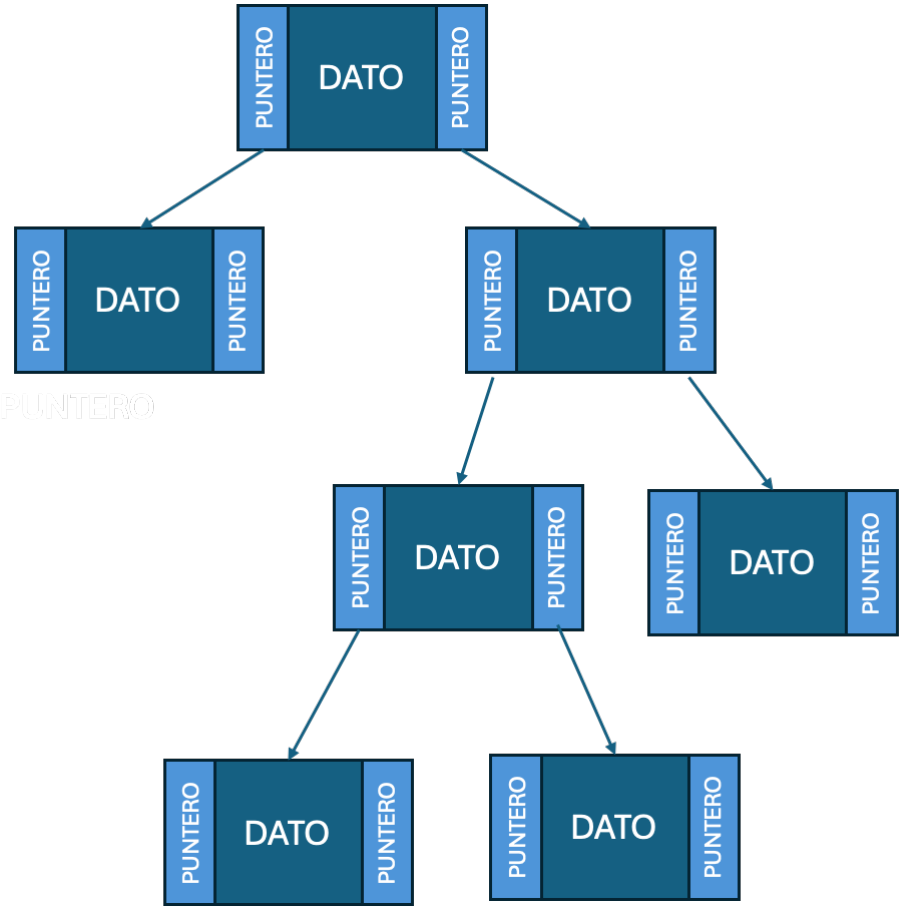
- **Representación Jerárquica:** Ideales para modelar estructuras con relaciones de uno a muchos.
- **Búsqueda Eficiente:** En árboles bien estructurados (como los BST), las operaciones de búsqueda, inserción y eliminación son muy rápidas (O(logN) en el caso promedio).
- **Ordenamiento:** Algunos tipos de árboles mantienen los datos ordenados.

Árboles binarios

Un **Árbol Binario** es un tipo especial de árbol en el que cada nodo tiene como máximo **dos hijos**, referidos como el **hijo izquierdo** y el **hijo derecho**.

Formalmente se define un árbol binario tipo T como una estructura homogénea, resultado de la concatenación de un elemento de tipo T,

llamado raíz, con dos árboles binarios disjuntos, llamados subárbol izquierdo y subárbol derecho. Un árbol binario especial es el árbol vacío.



Tipos Especiales de Árboles Binarios:

- **Árbol Binario Completo (Complete Binary Tree):** árbol en el que todos sus nodos, excepto los del último nivel, tienen dos hijos: el subárbol izquierdo y el subárbol derecho. En el caso de que el último nivel no tenga dos hijos, suele insertarse primero el de la izquierda.
- **Árbol Binario Lleno (Full Binary Tree / Strictly Binary Tree):** Cada nodo tiene 0 o 2 hijos. No hay nodos con un solo hijo.

- **Árbol Binario Perfecto (Perfect Binary Tree):** Es un árbol binario lleno en el que todas las hojas están en el mismo nivel y todos los nodos internos tienen exactamente dos hijos.
- **Árbol Binario Sesgado (Skewed Binary Tree):** Todos los nodos tienen solo un hijo, formando una estructura similar a una lista enlazada. Puede ser sesgado a la izquierda o a la derecha.

Árboles Binarios de Búsqueda (BST - Binary Search Trees)

Un **Árbol Binario de Búsqueda (BST)** es un tipo especial de árbol binario que organiza los nodos de manera que la búsqueda, inserción y eliminación de elementos sean muy eficientes. Su principal característica es que la información se almacena en los nodos cuidando de mantener cierto orden.

Formalmente se define un árbol binario de búsqueda de la siguiente manera: Para todo nodo T del árbol se debe cumplir que todos los valores almacenados en el subárbol izquierdo de T sean menores o iguales a la información guardada en el nodo T. De forma similar, todos los valores almacenados en el subárbol derecho de T deben ser mayores o iguales a la información guardada en el nodo T.

Propiedades Clave de un BST:

1. El valor de todos los nodos en el **subárbol izquierdo** de un nodo es **menor** que el valor del propio nodo.
2. El valor de todos los nodos en el **subárbol derecho** de un nodo es **mayor** que el valor del propio nodo.

3. Tanto el subárbol izquierdo como el derecho son, a su vez, también BSTs.
4. No hay nodos duplicados (aunque algunas implementaciones permiten duplicados).

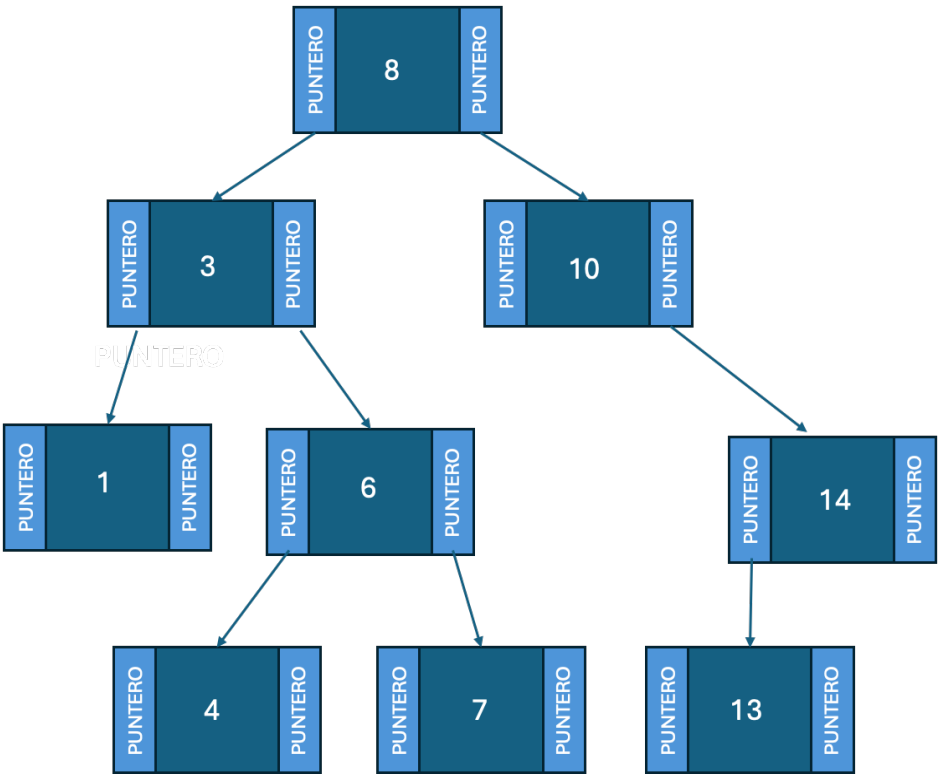
Operaciones Básicas en un BST:

- **Búsqueda (search(valor)):**
  - Comienza en la raíz.
  - Si el valor es igual al nodo actual, se encuentra.
  - Si el valor es menor, ve al subárbol izquierdo.
  - Si el valor es mayor, ve al subárbol derecho.
  - Si llegas a un nodo None (nulo), el valor no está en el árbol.
  - **Complejidad:** O(h), donde h es la altura del árbol. En el mejor caso (árbol balanceado), O(logN). En el peor caso (árbol sesgado), O(N).
- **Inserción (insert(valor)):**
  - Similar a la búsqueda, se desciende por el árbol.
  - Si el valor es menor, se va a la izquierda. Si es mayor, a la derecha.
  - Cuando se llega a un nodo None, se inserta el nuevo nodo en esa posición.
  - **Complejidad:** O(h).
- **Eliminación (delete(valor)):** Esta es la operación más compleja en un BST y tiene tres casos:
  - **El nodo a eliminar es una hoja:** Simplemente se elimina.

- **El nodo a eliminar tiene un hijo:** El nodo se reemplaza por su único hijo.
- **El nodo a eliminar tiene dos hijos:** Se debe encontrar un "sustituto" que mantenga la propiedad del BST. Comúnmente, se usa el **sucesor inorden** (el nodo más pequeño en el subárbol derecho) o el **predecesor inorden** (el nodo más grande en el subárbol izquierdo). El nodo a eliminar se reemplaza con este sucesor/predecesor, y luego se elimina el sucesor/predecesor de su posición original.

Recorridos de Árboles Binarios

Un recorrido (o transversal) es un proceso de visitar cada nodo en el árbol exactamente una vez. Hay cuatro métodos comunes:



1. Recorrido en Preorden (Preorder Traversal - Raíz-Izquierda-Derecha)

- **Visita** el nodo actual (Raíz).
- **Recorre** el subárbol izquierdo.
- **Recorre** el subárbol derecho.

Uso: Para crear una copia de un árbol, o para expresiones (prefijo).

Ejemplo (árbol anterior):

Recorrido Preorden: 8, 3, 1, 6, 4, 7, 10, 14, 13

2. Recorrido en Inorden (Inorder Traversal - Izquierda-Raíz-Derecha)

- **Recorre** el subárbol izquierdo.
- **Visita** el nodo actual (Raíz).
- **Recorre** el subárbol derecho.

Uso: En un BST, este recorrido devuelve los elementos en orden ascendente (ordenado).

Ejemplo (árbol anterior):

Recorrido Inorden: 1, 3, 4, 6, 7, 8, 10, 13, 14

3. Recorrido en Postorden (Postorder Traversal - Izquierda-Derecha-Raíz)

- **Recorre** el subárbol izquierdo.
- **Recorre** el subárbol derecho.
- **Visita** el nodo actual (Raíz).

Uso: Para eliminar un árbol (liberar memoria), o para expresiones (postfijo).

Ejemplo (árbol anterior):

Recorrido Postorden: 1, 4, 7, 6, 3, 13, 14, 10, 8

4. Recorrido en Anchura (Level Order Traversal)

- Visita los nodos nivel por nivel, de izquierda a derecha. Se suele implementar con una cola.

Uso: Búsqueda del camino más corto, árboles de sufijos.

Ejemplo (árbol anterior):

Recorrido por Niveles: 8, 3, 10, 1, 6, 14, 4, 7, 13

Aplicaciones Comunes de los Árboles

- **Sistemas de Archivos:** La estructura de directorios y archivos en un sistema operativo es un árbol (cada directorio puede tener subdirectorios y archivos).
- **Bases de Datos:** Los índices (como B-trees y B+-trees) se utilizan para acelerar las consultas.
- **Análisis Sintáctico (Parsing):** Los compiladores construyen árboles de sintaxis abstracta (AST) para representar la estructura del código.
- **Inteligencia Artificial:** Árboles de decisión para clasificación y toma de decisiones.

- **Juegos:** Árboles de juego para representar posibles movimientos y resultados.
- **Algoritmos de Enrutamiento en Redes:** Representación de topologías de red.
- **XML/HTML Parsing:** El DOM (Document Object Model) es una estructura de árbol.
- **Estructuras de Directorios (LDAP, Active Directory).**